Queues and workers

There's only one thing left to do. If remember I said in the introduction that if you use only one queue for everything you'll have some problematic situations. I'm not going to repeat it, but imagine there are 10,000 notifications in the queue, and you dispatch the payout jobs.

To avoid these situations all you need to do is to use multiple queues. In this project, we want to have a dedicated `payout` queue that has only payout jobs. Basically, we only need to instruct Laravel that it should dispatch the payout jobs on a `payout` queue:

```
class SendPayoutJob implements ShouldQueue
{
  public function __construct(
      private readonly User $user,
      private readonly TransactionReport $report,
      private readonly string $uuid,
      private readonly DateInterval $period,
  ) {
      $this->onQueue('payout');
  }
}
```

`$this->onQueue('payout')` ensures that this job is always dispatched onto the `payout` queue.

The next step is to start a worker that processes the `payout` queue. There's a `--queue` flag in the `queue:work` command so we can do this: `php artisan queue:work --queue=payout` And there can be another worker process that processes the default queue: `php artisan queue:work --queue=default` You could think that it's a good idea to have a dedicated worker only for the payout queue, right? After all, these are the most important jobs. But actually, it's not true. I'll show you why.

